Deponentlanadigan materiallarning titul varag'i

EHM uchuh dastur (Ma'lumotlar bazasi) nomi:

"MNAZORAT yagona axborot tizimi"

Huquq ega(lar)si

1. Yusupov Mansur Maksudovich

Muallif(lar)

1. Yusupov Mansur Maksudovich

EHM uchun dasturni identifikatsiya qiluvchi materiallar dastlabki matni (Dastur kodi)

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
import { Logger, ValidationPipe } from '@nestjs/common';
import * as cookieParser from 'cookie-parser';
import { PrismaClientExceptionFilter } from './common/exceptions/prisma-exception';
import * as cl from 'cluster';
import { cpus } from 'os';
import { NestExpressApplication } from '@nestjs/platform-express';
import { join } from 'path';
import * as express from 'express';
import * as process from 'node:process';

const cluster = cl as unknown as cl.Cluster;

async function bootstrap() {
  const app = await NestFactory.create<NestExpressApplication>(AppModule);
  app.setGlobalPrefix('api/v1');

  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      forbidNonWhitelisted: true,
      transform: true,
    }),
  );

  app.use(express.json({ limit: '50mb' }));
  app.use(express.urlencoded({ limit: '50mb', extended: true }));

  app.enableCors({
```

```
    credentials: true,
    origin: (origin, callback) => {
      const allowedOrigins = [
        'https://new.mnazorat.uz',
        'http://new.mnazorat.uz',
        'http://localhost:5173',
        'http://localhost:8001',
      ];

      if (!origin || allowedOrigins.includes(origin)) {
        callback(null, true);
      } else {
        callback(new Error('CORS policy violation'));
      }
    },
    methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS', 'PATCH'],
  });

  app.use('/api/v1/static', express.static(join(process.cwd(), 'static')));

  app.use(cookieParser());
  const httpAdapter = app.getHttpAdapter();
  app.useGlobalFilters(new PrismaClientExceptionFilter(httpAdapter));

  const port = process.env.PORT || 3000;
  await app.listen(port);
  return port;
}

let processCount = 0;
if (process.env.NODE_ENV === 'development') {
  bootstrap().then((port) => {
    Logger.log(`Backend is running on http://localhost:${port}/api/v1`, 'Bootstrap');
    Logger.log(`BullBoard is running on http://localhost:${port}/api/v1/queues`, 'Bootstrap');
```

```
  });
} else if (cluster.isPrimary) {
  const numCPUs = cpus().length;
  Logger.log(`Основной процесс запущен. Запускаем ${numCPUs} рабочих процессов...`);
  for (let i = 0; i < numCPUs; i++) {
    if (i === 0) {
      process.env['PROCESS_WORKER_ID'] = '0';
    }
    cluster.fork();
  }

  cluster.on('exit', (worker, code, signal) => {
    Logger.warn(`Рабочий процесс ${worker.process.pid} завершился. Перезапуск...`);
    cluster.fork();
  });
  processCount++;
} else {
  bootstrap().then((port) => {
    Logger.log(`Рабочий процесс ${process.pid} запущен`);
    Logger.log(`Backend is running on http://localhost:${port}/api/v1`, 'Bootstrap');
    Logger.log(`BullBoard is running on http://localhost:${port}/api/v1/queues`, 'Bootstrap');
  });
}

import { Module } from '@nestjs/common';
import { AuthGuard, PrismaModule } from './common';
import { BullModule } from '@nestjs/bullmq';
import { BullBoardModule } from '@bull-board/nestjs';
import { ExpressAdapter } from '@bull-board/express';
import { AuthModule } from './modules/auth/auth.module';
import { ConfigModule, ConfigService } from '@nestjs/config';
import { JwtModule } from '@nestjs/jwt';
import { CacheModule } from '@nestjs/cache-manager';
import { APP_GUARD } from '@nestjs/core';
```

```
import { EventEmitterModule } from '@nestjs/event-emitter';

import { TaskTypeModule } from './modules/task-type/task-type.module';

import { OrganizationModule } from './modules/organization/organization.module';

import { ReferencesModule } from './modules/references/references.module';

import { TaskStateModule } from './modules/task-state/task-state.module';

import { FaceIdModule } from './modules/face-id/face-id.module';

import { AttendanceModule } from './modules/attendance/attendance.module';

import { UserModule } from './modules/user/user.module';

import { RecipientAnswerTypeModule } from './modules/recipient-answer-type/recipient-answer-
type.module';

import { TaskModule } from './modules/task/task.module';

import { UserWorkingScheduleDayModule } from './modules/user-working-schedule-day/user-working-
schedule-day.module';

import { UserWorkingScheduleModule } from './modules/user-working-schedule/user-working-
schedule.module';

import { GpsModule } from './modules/gps/gps.module';

import { FileModule } from './modules/file/file.module';

import { ScheduleModule } from '@nestjs/schedule';

import { DashboardModule } from './modules/dashboard/dashboard.module';

import { RecipientModule } from './modules/recipient/recipient.module';

import * as cl from 'cluster';

import { AppController } from './app.controller';

import { CoordsModule } from './modules/coords/coords.module';

import { StatsModule } from './modules/stats/stats.module';

import { NotificationModule } from './modules/notification/notification.module';

import { DynamicModelModule } from './modules/dynamic-model/dynamic-model.module';

import { VacancyModule } from './modules/vacancy/vacancy.module';

import { VacationModule } from './modules/vacation/vacation.module';

import { BotModule } from './modules/telegram-bot/bot.module';

import { SectionStatsExcelModule } from './modules/dashboard/section-stats/excel/section-stats-
excel.module';

import { WorkerModule } from './modules/worker/worker.module';

import { createKeyv } from '@keyv/redis';

import { InfractionModule } from './modules/infraction/infraction.module';
```

```
import { InfractionReasonModule } from './modules/infraction-reason/infraction-reason.module';

const cluster = cl as unknown as cl.Cluster;
const isSingleProcess = process.env.NODE_ENV === 'development' || cluster.isPrimary;

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    ScheduleModule.forRoot(),
    PrismaModule,
    AuthModule,
    TaskTypeModule,
    TaskStateModule,
    RecipientAnswerTypeModule,
    OrganizationModule,
    TaskModule,
    UserWorkingScheduleDayModule,
    UserWorkingScheduleModule,
    DynamicModelModule,
    SectionStatsExcelModule,
    WorkerModule,
    InfractionReasonModule,
    JwtModule.registerAsync({
      global: true,
      useFactory: (configService: ConfigService) => ({
        global: true,
        secret: configService.get('JWT_SECRET'),
        signOptions: { expiresIn: '60s' },
      }),
      inject: [ConfigService],
    }),
    CacheModule.registerAsync({
      isGlobal: true,
      useFactory: (configService: ConfigService) => {
```

```
    const host = configService.get<string | undefined>('REDIS_HOST');
    const port = configService.get<string | undefined>('REDIS_PORT');
    const password = configService.get<string | undefined>('REDIS_PASSWORD_ENV');

    let redisUrl = 'redis://';

    if (host) {
      if (password) {
        redisUrl += `:${password}@${host}`;
      } else {
        redisUrl += host;
      }

      if (port) {
        redisUrl += `:${port}`;
      }
    } else {
      // Default values agar environment variables mavjud bo'lmasa
      redisUrl = 'redis://localhost:6379';
    }

    return {
      stores: [createKeyv(redisUrl)],
    };
  },
  inject: [ConfigService],
},
BullModule.forRootAsync({
  useFactory: async (configService: ConfigService) => ({
    connection: {
      host: configService.get<string>('REDIS_HOST'),
      port: configService.get<number>('REDIS_PORT'),
      password: configService.get<string>('REDIS_PASSWORD_ENV'),
    },
```

```
      }),
      inject: [ConfigService],
    }),
    BullBoardModule.forRoot({
      route: '/queues',
      adapter: ExpressAdapter,
    }),
    BotModule,
    EventEmitterModule.forRoot(),
    OrganizationModule,
    ReferencesModule,
    FaceIdModule,
    AttendanceModule,
    UserModule,
    GpsModule,
    FileModule,
    DashboardModule,
    RecipientModule,
    CoordsModule,
    StatsModule,
    NotificationModule,
    VacancyModule,
    VacationModule,
    InfractionModule,
  ],
  controllers: [AppController],
  providers: [
    {
      provide: APP_GUARD,
      useClass: AuthGuard,
    },
  ],
})
export class AppModule {}
```

```
import { Controller, Get } from '@nestjs/common';
import { ProtectedRoute } from './common';

@Controller()
export class AppController {
  @ProtectedRoute({ isPublic: true })
  @Get('health')
  getHealth() {
    return 'OK';
  }
}

import { Injectable } from '@nestjs/common';
import * as ExcelJS from 'exceljs';
import { join } from 'path';
import * as fs from 'fs';
import { AttendanceService } from './attendance.service';
import * as dayjs from 'dayjs';
import { TimezoneHelper, DEFAULT_TIMEZONE } from '../../helpers/timezone.helper';

// Dayjs setup bir marta bajariladi
dayjs.tz.setDefault(DEFAULT_TIMEZONE);

@Injectable()
export class AttendanceExcelService {
  constructor(private readonly attendanceService: AttendanceService) {}

  async generateAttendanceReportExcel(orgIds: string[], startDate?: string, endDate?: string):
Promise<string> {
    try {
      // Получаем данные отчета через метод getAttendanceReport
      const reportData = await this.attendanceService.getAttendanceReport(orgIds, startDate, endDate);
```

```
if (!reportData || !reportData.workers || reportData.workers.length === 0) {
  throw new Error("Hisobot ma'lumotlari topilmadi");
}

// Создаем новую рабочую книгу Excel
const workbook = new ExcelJS.Workbook();

// Устанавливаем метаданные документа
workbook.creator = 'Monitoring Nazorat';
workbook.lastModifiedBy = 'Monitoring Nazorat';
workbook.created = TimezoneHelper.now().toDate();
workbook.modified = TimezoneHelper.now().toDate();

// Создаем лист с подробным отчетом о посещаемости
this.createDetailedAttendanceReportSheet(
  workbook,
  reportData.workers,
  TimezoneHelper.formatLocal(reportData.reportPeriod.startDate, 'YYYY-MM-DD'),
  TimezoneHelper.formatLocal(reportData.reportPeriod.endDate, 'YYYY-MM-DD'),
);

// Создаем лист со сводным отчетом
this.createSummaryReportSheet(
  workbook,
  reportData.workers,
  TimezoneHelper.formatLocal(reportData.reportPeriod.startDate, 'YYYY-MM-DD'),
  TimezoneHelper.formatLocal(reportData.reportPeriod.endDate, 'YYYY-MM-DD'),
);

// Создаем папку для хранения файлов Excel, если она не существует
const excelDir = join(process.cwd(), 'uploads', 'excel');
if (!fs.existsSync(excelDir)) {
  fs.mkdirSync(excelDir, { recursive: true });
}
```

```
    // Сохраняем Excel-файл с текущей датой в названии
    const fileName = `davomat_hisoboti_${TimezoneHelper.now().format('YYYY-MM-DD_HH-mm-ss')}.xlsx`;
    const filePath = join(excelDir, fileName);

    await workbook.xlsx.writeFile(filePath);

    return filePath;
  } catch (error) {
    throw new Error(`Excel fayl yaratishda xatolik: ${error.message}`);
  }
}

async generateAttendanceReportByOrganizationTypePositionExcel(
  orgId: string,
  positionId: string,
  startDate?: string,
  endDate?: string,
  page: number = 1,
  limit: number = 20,
): Promise<string> {
  try {
    // Получаем данные отчета через метод getAttendanceReportByOrganizationTypePosition
    const reportData = await this.attendanceService.getAttendanceReportByOrganizationTypePosition(
      orgId,
      positionId,
      startDate,
      endDate,
      page,
      limit,
    );

    if (!reportData || !reportData.workers || reportData.workers.length === 0) {
```

```
  throw new Error("Hisobot ma'lumotlari topilmadi");
}

// Создаем новую рабочую книгу Excel
const workbook = new ExcelJS.Workbook();

// Устанавливаем метаданные документа
workbook.creator = 'Monitoring Nazorat';
workbook.lastModifiedBy = 'Monitoring Nazorat';
workbook.created = TimezoneHelper.now().toDate();
workbook.modified = TimezoneHelper.now().toDate();

// Создаем лист с подробным отчетом о посещаемости для конкретной позиции
this.createDetailedAttendanceReportSheetByPosition(
  workbook,
  reportData.workers,
  reportData.position,
  TimezoneHelper.formatLocal(reportData.reportPeriod.startDate, 'YYYY-MM-DD'),
  TimezoneHelper.formatLocal(reportData.reportPeriod.endDate, 'YYYY-MM-DD'),
);

// Создаем лист со сводным отчетом для конкретной позиции
this.createSummaryReportSheetByPosition(
  workbook,
  reportData.workers,
  reportData.position,
  TimezoneHelper.formatLocal(reportData.reportPeriod.startDate, 'YYYY-MM-DD'),
  TimezoneHelper.formatLocal(reportData.reportPeriod.endDate, 'YYYY-MM-DD'),
);

// Создаем папку для хранения файлов Excel, если она не существует
const excelDir = join(process.cwd(), 'uploads', 'excel');
if (!fs.existsSync(excelDir)) {
  fs.mkdirSync(excelDir, { recursive: true });
```

```
    }

    // Сохраняем Excel-файл с текущей датой в названии
    const fileName = `lavozim_davomat_hisoboti_${TimezoneHelper.now().format('YYYY-MM-DD_HH-mm-ss')}.xlsx`;
    const filePath = join(excelDir, fileName);

    await workbook.xlsx.writeFile(filePath);

    return filePath;
  } catch (error) {
    throw new Error(`Excel fayl yaratishda xatolik: ${error.message}`);
  }
}

private getAllUniqueDates(employeesData: any[], startDate: string, endDate: string): string[] {
  const datesSet = new Set<string>();

  // Барча ходимларнинг саналарини тўплаш
  employeesData.forEach((employee) => {
    employee.dailyReports.forEach((report) => {
      const dateKey = TimezoneHelper.formatLocal(report.date, 'YYYY-MM-DD');
      datesSet.add(dateKey);
    });
  });

  // Саналарни сортлаш
  const sortedDates = Array.from(datesSet).sort();
  return sortedDates;
}

private createDetailedAttendanceReportSheet(
  workbook: ExcelJS.Workbook,
  employeesData: any[],
```

```
    startDate: string,
    endDate: string,
  ) {
    // Создаем лист для подробного отчета
    const worksheet = workbook.addWorksheet('Batafsil hisobot', {
      properties: {
        tabColor: { argb: '2F75B5' },
        defaultRowHeight: 25, // Увеличили высоту строк с 20 до 25
      },
    });

    // Получаем все уникальные даты из отчетов
    const allDates = this.getAllUniqueDates(employeesData, startDate, endDate);

    // Заголовок отчета
    const titleRow = worksheet.addRow(['Xodimlarning Batafsil Davomat Hisoboti']);
    titleRow.font = { size: 16, bold: true };
    titleRow.height = 35;

    // Информация о периоде
    const periodRow = worksheet.addRow([`Davr: ${startDate} - ${endDate}`]);
    periodRow.font = { size: 12, bold: true };
    periodRow.height = 25;

    // Пустая строка для разделения
    worksheet.addRow([]);

    // Создаем заголовки - основные колонки плюс новые колонки времени плюс динамические
даты
    const baseHeaders = ['№', 'Tashkilot', 'F.I.O', 'Lavozim', 'Reja vaqt\n(umumiy)', 'Fakt
vaqt\n(umumiy)'];

    // Сначала добавляем заголовки дат (без подзаголовков)
    const dateHeaders = [];
```

```
allDates.forEach((date) => {
  const formattedDate = dayjs(date).format('DD.MM');
  const dayName = this.getDayName(dayjs(date).day());
  dateHeaders.push(`${formattedDate}\n${dayName}`);
  dateHeaders.push(''); // Пустые для объединения
  dateHeaders.push(''); // Пустые для объединения
});

const allHeaders = [...baseHeaders, ...dateHeaders];
const headerRow = worksheet.addRow(allHeaders);

// Создаем второй ряд заголовков для подзаголовков
const subHeaders = ['', '', '', '', '', ''']; // Пустые для основных колонок (6 штук)
allDates.forEach(() => {
  subHeaders.push('Kelish', 'Ketish', 'Ishlagan');
});
const subHeaderRow = worksheet.addRow(subHeaders);

// Стили для основных заголовков
headerRow.eachCell((cell, colNumber) => {
  cell.font = { bold: true, size: 14, color: { argb: 'FFFFFF' } }; // Увеличили размер шрифта заголовков
  cell.fill = {
    type: 'pattern',
    pattern: 'solid',
    fgColor: { argb: '2F75B5' },
  };
  cell.alignment = { horizontal: 'center', vertical: 'middle', wrapText: true };
  cell.border = {
    top: { style: 'thin' },
    left: { style: 'thin' },
    bottom: { style: 'thin' },
    right: { style: 'thin' },
  };
```

```
    // Устанавливаем ширину колонок
    if (colNumber <= 6) {
      // Основные колонки
      const widths = [5, 20, 25, 18, 15, 15]; // Қўшилган 2 та вақт устуни учун кенглик
      worksheet.getColumn(colNumber).width = widths[colNumber - 1];
    } else {
      // Колонки дат - делаем шире для 3 подколонок (увеличили с 6 до 12 для лучшей читаемости)
      worksheet.getColumn(colNumber).width = 12;
    }
  });

  // Стили для подзаголовков
  subHeaderRow.eachCell((cell, colNumber) => {
    if (colNumber > 6) {
      cell.font = { bold: true, size: 12, color: { argb: 'FFFFFF' } }; // Увеличили размер шрифта
подзаголовков с 9 до 12

      // Определяем тип подколонки для элегантных цветов
      const subColIndex = (colNumber - 7) % 3;
      let bgColor = '2F75B5'; // По умолчанию синий

      if (subColIndex === 0) {
        bgColor = '6C9BD1'; // Мягкий синий для "Kelish"
      } else if (subColIndex === 1) {
        bgColor = 'E8A87C'; // Мягкий оранжевый для "Ketish"
      } else {
        bgColor = '98D8C8'; // Мягкий зеленый для "Ishlagan"
      }

      cell.fill = {
        type: 'pattern',
        pattern: 'solid',
        fgColor: { argb: bgColor },
```

```
    };
    cell.alignment = { horizontal: 'center', vertical: 'middle' };
    cell.border = {
      top: { style: 'thin' },
      left: { style: 'thin' },
      bottom: { style: 'thin' },
      right: { style: 'thin' },
    };
  } else {
    // Для основных колонок делаем такой же стиль как основной заголовок
    cell.font = { bold: true, size: 14, color: { argb: 'FFFFFF' } }; // Увеличили размер шрифта
    cell.fill = {
      type: 'pattern',
      pattern: 'solid',
      fgColor: { argb: '2F75B5' },
    };
    cell.border = {
      top: { style: 'thin' },
      left: { style: 'thin' },
      bottom: { style: 'thin' },
      right: { style: 'thin' },
    };
  }
});

headerRow.height = 35; // Увеличили высоту заголовков с 30 до 35
subHeaderRow.height = 30; // Увеличили высоту подзаголовков с 25 до 30

// Объединяем ячейки для заголовка и периода
const totalColumns = baseHeaders.length + allDates.length * 3;
worksheet.mergeCells(1, 1, 1, totalColumns);
worksheet.mergeCells(2, 1, 2, totalColumns);

// Объединяем ячейки для дат (каждая дата занимает 3 колонки)
```

```
let currentCol = 7; // Начинаем после основных колонок (6 шт)
allDates.forEach(() => {
  worksheet.mergeCells(4, currentCol, 4, currentCol + 2); // Объединяем 3 колонки для каждой даты
  currentCol += 3;
});

// Объединяем основные заголовки по вертикали
worksheet.mergeCells(4, 1, 5, 1); // №
worksheet.mergeCells(4, 2, 5, 2); // Tashkilot
worksheet.mergeCells(4, 3, 5, 3); // F.I.O
worksheet.mergeCells(4, 4, 5, 4); // Lavozim
worksheet.mergeCells(4, 5, 5, 5); // Reja vaqt (umumiy)
worksheet.mergeCells(4, 6, 5, 6); // Fakt vaqt (umumiy)

// Добавляем данные по каждому сотруднику
employeesData.forEach((employee, index) => {
  // Вычисляем общее время
  let totalPlanMinutes = 0;
  let totalFactMinutes = 0;

  employee.dailyReports.forEach((report) => {
    if (report.isWorkDay) {
      // Считаем плановое время (например, 8 часов = 480 минут)
      totalPlanMinutes += 480; // Можно изменить в зависимости от требований

      // Считаем фактическое время
      if (report.minutesWorked > 0) {
        totalFactMinutes += report.minutesWorked;
      }
    }
  });

  const rowData = [
    index + 1,
```

```
      employee.organization?.name || '-',
      employee.employee?.fullName || '-',
      employee.employee?.position?.name || '-',
      this.formatMinutesToTime(totalPlanMinutes), // Reja vaqt (umumiy)
      this.formatMinutesToTime(totalFactMinutes), // Fakt vaqt (umumiy)
    ];

    // Создаем карту отчетов по датам для быстрого поиска
    const reportsByDate = new Map();
    employee.dailyReports.forEach((report) => {
      const dateKey = TimezoneHelper.formatLocal(report.date, 'YYYY-MM-DD');
      reportsByDate.set(dateKey, report);
    });

    // Добавляем данные для каждой даты
    allDates.forEach((date) => {
      const dateKey = dayjs(date).format('YYYY-MM-DD');
      const report = reportsByDate.get(dateKey);

      if (report) {
        // Проверяем статус дня
        if (!report.isWorkDay) {
          // Если не рабочий день - объединяем в одну ячейку "Dam"
          rowData.push('Dam', '', '');
        } else if (!report.cameToWork) {
          // Если не пришел на работу - объединяем в одну ячейку "Yo'q"
          rowData.push("Yo'q", '', '');
        } else {
          // Обычный рабочий день с данными
          const enterTime = report.actualEnterTime ? TimezoneHelper.formatTime(report.actualEnterTime)
: '-';
          const exitTime = report.actualExitTime ? TimezoneHelper.formatTime(report.actualExitTime) : '-
';
          const workTime = report.minutesWorked > 0 ?
this.formatMinutesToTime(report.minutesWorked) : '0:00';
```

```
      rowData.push(enterTime, exitTime, workTime);
    }
  } else {
    // Если нет данных для этой даты
    rowData.push('-', '-', '-');
  }
});

const dataRow = worksheet.addRow(rowData);

// Настраиваем стили для строки данных
dataRow.eachCell((cell, colNumber) => {
  cell.border = {
    top: { style: 'thin' },
    left: { style: 'thin' },
    bottom: { style: 'thin' },
    right: { style: 'thin' },
  };
  cell.alignment = { vertical: 'middle', horizontal: 'center' };
  cell.font = { size: 12, bold: true }; // Увеличили размер шрифта с 8 до 12

  // Применяем цвета для дат в зависимости от статуса
  if (colNumber > 6) {
    const dateIndex = Math.floor((colNumber - 7) / 3);
    const dateKey = dayjs(allDates[dateIndex]).format('YYYY-MM-DD');
    const report = reportsByDate.get(dateKey);

    // Определяем базовый цвет для типа столбца
    const subColIndex = (colNumber - 7) % 3;
    let baseColor = '';

    if (subColIndex === 0) {
      baseColor = '6C9BD1'; // Мягкий синий для "Kelish"
```

```
      } else if (subColIndex === 1) {
        baseColor = 'E8A87C'; // Мягкий оранжевый для "Ketish"
      } else {
        baseColor = '98D8C8'; // Мягкий зеленый для "Ishlagan"
      }

      // Если нет данных - используем светлую версию базового цвета
      const lightColor = this.lightenColor(baseColor);
      cell.fill = {
        type: 'pattern',
        pattern: 'solid',
        fgColor: { argb: lightColor },
      };
    }
  });

  // Объединяем ячейки для "Yo'q" и "Dam"
  allDates.forEach((date, dateIndex) => {
    const dateKey = dayjs(date).format('YYYY-MM-DD');
    const report = reportsByDate.get(dateKey);

    if (report) {
      if (!report.isWorkDay || !report.cameToWork) {
        // Объединяем 3 колонки для "Dam" или "Yo'q"
        const startCol = 7 + dateIndex * 3; // 7 - начало колонок дат
        const endCol = startCol + 2;
        worksheet.mergeCells(dataRow.number, startCol, dataRow.number, endCol);
      }
    }
  });
});

// Закрепляем заголовок и первые 6 колонок
worksheet.views = [{ state: 'frozen', xSplit: 6, ySplit: 5 }];
```

```
}

private createSummaryReportSheet(
  workbook: ExcelJS.Workbook,
  employeesData: any[],
  startDate: string,
  endDate: string,
) {
  // Создаем лист для сводного отчета
  const worksheet = workbook.addWorksheet('Umumiy hisobot', {
    properties: {
      tabColor: { argb: '4472C4' },
      defaultRowHeight: 25, // Увеличили высоту строк с 20 до 25
    },
  });

  // Заголовок отчета
  const titleRow = worksheet.addRow(['Xodimlarning Umumiy Davomat Hisoboti']);
  titleRow.font = { size: 16, bold: true };
  titleRow.height = 35;

  // Информация о периоде
  const periodRow = worksheet.addRow([`Davr: ${startDate} - ${endDate}`]);
  periodRow.font = { size: 12, bold: true };
  periodRow.height = 25;

  // Пустая строка для разделения
  worksheet.addRow([]);

  // Создаем заголовки таблицы
  const headerRow = worksheet.addRow([
    '№',
    'Tashkilot',
    'F.I.O',
```

```
    'Lavozim',
    'Ish kunlari',
    'Kelgan kunlar',
    'Davomat (foiz)',
    'Reja vaqt (soat)',
    'Fakt vaqt (soat)',
    'Kechikishlar soni',
    'Kechikish (minut)',
    'Erta ketishlar soni',
    'Erta ketish (minut)',
  ]);

  // Стили для заголовков
  headerRow.eachCell((cell) => {
    cell.font = { bold: true, size: 14, color: { argb: 'FFFFFF' } }; // Увеличили размер шрифта
заголовков сводки
    cell.fill = {
      type: 'pattern',
      pattern: 'solid',
      fgColor: { argb: '4472C4' },
    };
    cell.alignment = { horizontal: 'center', vertical: 'middle', wrapText: true };
    cell.border = {
      top: { style: 'thin' },
      left: { style: 'thin' },
      bottom: { style: 'thin' },
      right: { style: 'thin' },
    };
  });
  headerRow.height = 35; // Увеличили высоту заголовков сводки с 30 до 35

  // Объединяем ячейки для заголовка и периода
  worksheet.mergeCells(1, 1, 1, 13);
  worksheet.mergeCells(2, 1, 2, 13);
```

```
// Добавляем сводные данные по сотрудникам
employeesData.forEach((employee, index) => {
  const summary = employee.summary;

  // Вычисляем количество опозданий и ранних уходов
  const lateCount = employee.dailyReports.filter((report) => report.isLate).length;
  const earlyExitCount = employee.dailyReports.filter((report) => report.isEarlyExit).length;

  // Добавляем строку с данными сотрудника
  const dataRow = worksheet.addRow([
    index + 1,
    employee.organization?.name || '-',
    employee.employee?.fullName || '-',
    employee.employee?.position?.name || '-',
    summary.totalWorkDays || 0,
    summary.totalAttendanceDays || 0,
    `${summary.attendanceRate ? summary.attendanceRate.toFixed(1) : 0}%`,
    summary.planHours || '0:00',
    summary.factHours || '0:00',
    lateCount,
    summary.totalLateMinutes || 0,
    earlyExitCount,
    summary.totalEarlyExitMinutes || 0,
  ]);

  // Настраиваем стили для строки данных
  dataRow.eachCell((cell) => {
    cell.border = {
      top: { style: 'thin' },
      left: { style: 'thin' },
      bottom: { style: 'thin' },
      right: { style: 'thin' },
    };
```

```
    cell.alignment = { vertical: 'middle', horizontal: 'center' };
    cell.font = { size: 11, bold: false }; // Добавили увеличенный размер шрифта для данных
  });

  // Выделяем цветом строки сотрудников с низкой посещаемостью
  if (summary.attendanceRate < 80) {
    dataRow.eachCell((cell) => {
      cell.fill = {
        type: 'pattern',
        pattern: 'solid',
        fgColor: { argb: 'FFC7CE' }, // Светло-красный для низкой посещаемости
      };
    });
  }
});

// Устанавливаем ширину колонок
worksheet.getColumn(1).width = 5; // №
worksheet.getColumn(2).width = 25; // Tashkilot
worksheet.getColumn(3).width = 30; // F.I.O.
worksheet.getColumn(4).width = 20; // Lavozim
worksheet.getColumn(5).width = 15; // Ish kunlari
worksheet.getColumn(6).width = 15; // Kelgan kunlar
worksheet.getColumn(7).width = 15; // Davomat (foiz)
worksheet.getColumn(8).width = 15; // Reja vaqt
worksheet.getColumn(9).width = 15; // Fakt vaqt
worksheet.getColumn(10).width = 15; // Kechikishlar soni
worksheet.getColumn(11).width = 15; // Kechikish (minut)
worksheet.getColumn(12).width = 15; // Erta ketishlar soni
worksheet.getColumn(13).width = 15; // Erta ketish (minut)

// Закрепляем заголовок
worksheet.views = [{ state: 'frozen', xSplit: 0, ySplit: 4 }];
```

```
}

private createDetailedAttendanceReportSheetByPosition(
  workbook: ExcelJS.Workbook,
  employeesData: any[],
  position: any,
  startDate: string,
  endDate: string,
) {
  // Создаем лист для подробного отчета
  const worksheet = workbook.addWorksheet('Batafsil hisobot', {
    properties: {
      tabColor: { argb: '2F75B5' },
      defaultRowHeight: 25,
    },
  });

  // Получаем все уникальные даты из отчетов
  const allDates = this.getAllUniqueDates(employeesData, startDate, endDate);

  // Заголовок отчета
  const titleRow = worksheet.addRow([`Lavozim: ${position.name} - Batafsil Davomat Hisoboti`]);
  titleRow.font = { size: 16, bold: true };
  titleRow.height = 35;

  // Информация о периоде
  const periodRow = worksheet.addRow([`Davr: ${startDate} - ${endDate}`]);
  periodRow.font = { size: 12, bold: true };
  periodRow.height = 25;

  // Пустая строка для разделения
  worksheet.addRow([]);

  // Создаем заголовки - основные колонки плюс динамические даты
```

```
const baseHeaders = ['№', 'Tashkilot', 'F.I.O', 'Telefon', 'Reja vaqt\n(umumiy)', 'Fakt vaqt\n(umumiy)'];

// Добавляем заголовки дат
const dateHeaders = [];
allDates.forEach((date) => {
  const formattedDate = dayjs(date).format('DD.MM');
  const dayName = this.getDayName(dayjs(date).day());
  dateHeaders.push(`${formattedDate}\n${dayName}`);
  dateHeaders.push(''); // Пустые для объединения
  dateHeaders.push(''); // Пустые для объединения
});

const allHeaders = [...baseHeaders, ...dateHeaders];
const headerRow = worksheet.addRow(allHeaders);

// Создаем второй ряд заголовков для подзаголовков
const subHeaders = ['', '', '', '', '']; // Пустые для основных колонок
allDates.forEach(() => {
  subHeaders.push('Kelish', 'Ketish', 'Ishlagan');
});
const subHeaderRow = worksheet.addRow(subHeaders);

// Стили для основных заголовков
headerRow.eachCell((cell, colNumber) => {
  cell.font = { bold: true, size: 14, color: { argb: 'FFFFFF' } };
  cell.fill = {
    type: 'pattern',
    pattern: 'solid',
    fgColor: { argb: '2F75B5' },
  };
  cell.alignment = { horizontal: 'center', vertical: 'middle', wrapText: true };
  cell.border = {
    top: { style: 'thin' },
    left: { style: 'thin' },
```

```
    bottom: { style: 'thin' },
    right: { style: 'thin' },
  };

  // Устанавливаем ширину колонок
  if (colNumber <= 6) {
    const widths = [5, 20, 25, 15, 15, 15];
    worksheet.getColumn(colNumber).width = widths[colNumber - 1];
  } else {
    worksheet.getColumn(colNumber).width = 12;
  }
});

// Стили для подзаголовков
subHeaderRow.eachCell((cell, colNumber) => {
  if (colNumber > 6) {
    cell.font = { bold: true, size: 12, color: { argb: 'FFFFFF' } };

    const subColIndex = (colNumber - 7) % 3;
    let bgColor = '2F75B5';

    if (subColIndex === 0) {
      bgColor = '6C9BD1'; // Синий для "Kelish"
    } else if (subColIndex === 1) {
      bgColor = 'E8A87C'; // Оранжевый для "Ketish"
    } else {
      bgColor = '98D8C8'; // Зеленый для "Ishlagan"
    }

    cell.fill = {
      type: 'pattern',
      pattern: 'solid',
      fgColor: { argb: bgColor },
    };
```

```
    cell.alignment = { horizontal: 'center', vertical: 'middle' };
    cell.border = {
      top: { style: 'thin' },
      left: { style: 'thin' },
      bottom: { style: 'thin' },
      right: { style: 'thin' },
    };
  } else {
    cell.font = { bold: true, size: 14, color: { argb: 'FFFFFF' } };
    cell.fill = {
      type: 'pattern',
      pattern: 'solid',
      fgColor: { argb: '2F75B5' },
    };
    cell.border = {
      top: { style: 'thin' },
      left: { style: 'thin' },
      bottom: { style: 'thin' },
      right: { style: 'thin' },
    };
  }
});

headerRow.height = 35;
subHeaderRow.height = 30;

// Объединяем ячейки для заголовка и периода
const totalColumns = baseHeaders.length + allDates.length * 3;
worksheet.mergeCells(1, 1, 1, totalColumns);
worksheet.mergeCells(2, 1, 2, totalColumns);

// Объединяем ячейки для дат
let currentCol = 7;
allDates.forEach(() => {
```

```
  worksheet.mergeCells(4, currentCol, 4, currentCol + 2);
  currentCol += 3;
});

// Объединяем основные заголовки по вертикали
worksheet.mergeCells(4, 1, 5, 1); // №
worksheet.mergeCells(4, 2, 5, 2); // Tashkilot
worksheet.mergeCells(4, 3, 5, 3); // F.I.O
worksheet.mergeCells(4, 4, 5, 4); // Telefon
worksheet.mergeCells(4, 5, 5, 5); // Reja vaqt
worksheet.mergeCells(4, 6, 5, 6); // Fakt vaqt

// Добавляем данные по каждому сотруднику
employeesData.forEach((employee, index) => {
  const rowData = [
    index + 1,
    employee.organization?.name || employee.mainOrganization?.name || '-',
    employee.employee?.fullName || '-',
    employee.employee?.phone || '-',
    employee.summary?.planHours || '0:00',
    employee.summary?.factHours || '0:00',
  ];

  // Создаем карту отчетов по датам
  const reportsByDate = new Map();
  employee.dailyReports.forEach((report) => {
    const dateKey = TimezoneHelper.formatLocal(report.date, 'YYYY-MM-DD');
    reportsByDate.set(dateKey, report);
  });

  // Добавляем данные для каждой даты
  allDates.forEach((date) => {
    const dateKey = dayjs(date).format('YYYY-MM-DD');
    const report = reportsByDate.get(dateKey);
```

```
      if (report) {
        if (!report.isWorkDay) {
          rowData.push('Dam', '', '');
        } else if (!report.cameToWork) {
          rowData.push("Yo'q", '', '');
        } else {
          const enterTime = report.actualEnterTime ? TimezoneHelper.formatTime(report.actualEnterTime)
: '-';
          const exitTime = report.actualExitTime ? TimezoneHelper.formatTime(report.actualExitTime) : '-
';
          const workTime = report.minutesWorked > 0 ?
this.formatMinutesToTime(report.minutesWorked) : '0:00';
          rowData.push(enterTime, exitTime, workTime);
        }
      } else {
        rowData.push('-', '-', '-');
      }
    });

    const dataRow = worksheet.addRow(rowData);

    // Настройка стилей для строки данных
    dataRow.eachCell((cell, colNumber) => {
      cell.border = {
        top: { style: 'thin' },
        left: { style: 'thin' },
        bottom: { style: 'thin' },
        right: { style: 'thin' },
      };
      cell.alignment = { vertical: 'middle', horizontal: 'center' };
      cell.font = { size: 12, bold: true };

      if (colNumber > 6) {
        const dateIndex = Math.floor((colNumber - 7) / 3);
```

```
    const dateKey = dayjs(allDates[dateIndex]).format('YYYY-MM-DD');
    const report = reportsByDate.get(dateKey);

    const subColIndex = (colNumber - 7) % 3;
    let baseColor = '';

    if (subColIndex === 0) {
      baseColor = '6C9BD1';
    } else if (subColIndex === 1) {
      baseColor = 'E8A87C';
    } else {
      baseColor = '98D8C8';
    }

    const lightColor = this.lightenColor(baseColor);
    cell.fill = {
      type: 'pattern',
      pattern: 'solid',
      fgColor: { argb: lightColor },
    };
  }
});

// Объединяем ячейки для "Yo'q" и "Dam"
allDates.forEach((date, dateIndex) => {
  const dateKey = dayjs(date).format('YYYY-MM-DD');
  const report = reportsByDate.get(dateKey);

  if (report) {
    if (!report.isWorkDay || !report.cameToWork) {
      const startCol = 7 + dateIndex * 3;
      const endCol = startCol + 2;
      worksheet.mergeCells(dataRow.number, startCol, dataRow.number, endCol);
    }
```

```
      }
    });
  });

  // Закрепляем заголовок и первые 6 колонок
  worksheet.views = [{ state: 'frozen', xSplit: 6, ySplit: 5 }];
}

private createSummaryReportSheetByPosition(
  workbook: ExcelJS.Workbook,
  employeesData: any[],
  position: any,
  startDate: string,
  endDate: string,
) {
  // Создаем лист для сводного отчета
  const worksheet = workbook.addWorksheet('Umumiy hisobot', {
    properties: {
      tabColor: { argb: '4472C4' },
      defaultRowHeight: 25,
    },
  });

  // Заголовок отчета
  const titleRow = worksheet.addRow([`Lavozim: ${position.name} - Umumiy Davomat Hisoboti`]);
  titleRow.font = { size: 16, bold: true };
  titleRow.height = 35;

  // Информация о периоде
  const periodRow = worksheet.addRow([`Davr: ${startDate} - ${endDate}`]);
  periodRow.font = { size: 12, bold: true };
  periodRow.height = 25;

  // Пустая строка для разделения
```

```
worksheet.addRow([]);

// Заголовки для сводного отчета
const summaryHeaders = [
  '№',
  'Tashkilot',
  'F.I.O',
  'Telefon',
  'Umumiy ish kunlari',
  'Kelgan kunlar',
  'Kechikishlar (daqiqa)',
  'Erta ketishlar (daqiqa)',
  'Davomat foizi (%)',
  'Reja vaqt',
  'Fakt vaqt',
];

const summaryHeaderRow = worksheet.addRow(summaryHeaders);

// Стили для заголовков
summaryHeaderRow.eachCell((cell, colNumber) => {
  cell.font = { bold: true, size: 12, color: { argb: 'FFFFFF' } };
  cell.fill = {
    type: 'pattern',
    pattern: 'solid',
    fgColor: { argb: '4472C4' },
  };
  cell.alignment = { horizontal: 'center', vertical: 'middle', wrapText: true };
  cell.border = {
    top: { style: 'thin' },
    left: { style: 'thin' },
    bottom: { style: 'thin' },
    right: { style: 'thin' },
  };
```

```
  // Устанавливаем ширину колонок
  const widths = [5, 20, 25, 15, 12, 12, 15, 15, 12, 12, 12];
  worksheet.getColumn(colNumber).width = widths[colNumber - 1];
});

summaryHeaderRow.height = 30;

// Объединяем ячейки для заголовка и периода
worksheet.mergeCells(1, 1, 1, summaryHeaders.length);
worksheet.mergeCells(2, 1, 2, summaryHeaders.length);

// Добавляем данные сводного отчета
employeesData.forEach((employee, index) => {
  const summaryData = [
    index + 1,
    employee.organization?.name || employee.mainOrganization?.name || '-',
    employee.employee?.fullName || '-',
    employee.employee?.phone || '-',
    employee.summary?.totalWorkDays || 0,
    employee.summary?.totalAttendanceDays || 0,
    employee.summary?.totalLateMinutes || 0,
    employee.summary?.totalEarlyExitMinutes || 0,
    employee.summary?.attendanceRate ? employee.summary.attendanceRate.toFixed(1) : '0.0',
    employee.summary?.planHours || '0:00',
    employee.summary?.factHours || '0:00',
  ];

  const summaryRow = worksheet.addRow(summaryData);

  // Стили для данных
  summaryRow.eachCell((cell, colNumber) => {
    cell.border = {
      top: { style: 'thin' },
```

```
      left: { style: 'thin' },
      bottom: { style: 'thin' },
      right: { style: 'thin' },
    };
    cell.alignment = { vertical: 'middle', horizontal: 'center' };
    cell.font = { size: 11 };

    // Цветовое кодирование для процента посещаемости
    if (colNumber === 9) {
      const attendanceRate = parseFloat(cell.value as string);
      if (attendanceRate >= 90) {
        cell.fill = {
          type: 'pattern',
          pattern: 'solid',
          fgColor: { argb: 'C6EFCE' }, // Зеленый
        };
      } else if (attendanceRate >= 70) {
        cell.fill = {
          type: 'pattern',
          pattern: 'solid',
          fgColor: { argb: 'FFEB9C' }, // Желтый
        };
      } else {
        cell.fill = {
          type: 'pattern',
          pattern: 'solid',
          fgColor: { argb: 'FFC7CE' }, // Красный
        };
      }
    }
  });
});

// Закрепляем заголовки
```

```
    worksheet.views = [{ state: 'frozen', xSplit: 0, ySplit: 4 }];
  }

  private formatMinutesToTime(minutes: number | null | undefined): string {
    if (!minutes || minutes < 0) return '0:00';

    const hours = Math.floor(minutes / 60);
    const mins = minutes % 60;
    return `${hours.toString().padStart(2, '0')}:${mins.toString().padStart(2, '0')}`;
  }

  private getDayName(dayNumber: number): string {
    const days = ['Yakshanba', 'Dushanba', 'Seshanba', 'Chorshanba', 'Payshanba', 'Juma', 'Shanba'];
    return days[dayNumber] || "Noma'lum";
  }

  private lightenColor(hexColor: string): string {
    // Hex rangni RGB ga o'tkazamiz
    const r = parseInt(hexColor.substr(0, 2), 16);
    const g = parseInt(hexColor.substr(2, 2), 16);
    const b = parseInt(hexColor.substr(4, 2), 16);

    // Rangni 30% yorqinlashtitamiz
    const factor = 0.3;
    const newR = Math.min(255, Math.floor(r + (255 - r) * factor));
    const newG = Math.min(255, Math.floor(g + (255 - g) * factor));
    const newB = Math.min(255, Math.floor(b + (255 - b) * factor));

    // Yana hex formatga qaytaramiz
    return newR.toString(16).padStart(2, '0') + newG.toString(16).padStart(2, '0') +
newB.toString(16).padStart(2, '0');
  }
}
```

```
import { BadRequestException, Injectable, Logger } from '@nestjs/common';
import { Processor, WorkerHost } from '@nestjs/bullmq';
import { ATTENDANCE_REPORT } from './interface/attendance';
import { Job } from 'bullmq';
import { AttendanceType, Status } from '@prisma/client';
import { AttendanceHandler } from './attendance.handler';

interface AttendanceReportJobData {
  id: string;
  userId: string;
  status: Status;
  organizationId: string;
  time: Date;
  createdById: string | null;
  description: string | null;
  type: AttendanceType;
  createdAt: Date;
  updatedAt: Date;
}

type AttendanceReportJobName = 'attendance.ENTER' | 'attendance.EXIT';

@Injectable()
@Processor(ATTENDANCE_REPORT, {
  concurrency: 2,
  limiter: {
    max: 50,
    duration: 1000,
  },
})
export class AttendanceReportProcessor extends WorkerHost {
  logger = new Logger(AttendanceReportProcessor.name);

  constructor(private readonly attendanceHandler: AttendanceHandler) {
```

```
    super();
    this.logger.log('AttendanceReportProcessor initialized');
  }

  async process(job: Job<AttendanceReportJobData, unknown, AttendanceReportJobName>) {
    this.logger.debug('Processing attendance report', job.name);
    const jobHandler = await this.jobSwitcher(job);
    if (!jobHandler) {
      this.logger.error(`Job handler for ${job.name} not found`);
      throw new BadRequestException(`Job handler for ${job.name} not found`);
    }
  }

  private async jobSwitcher(job: Job<AttendanceReportJobData, unknown, AttendanceReportJobName>)
{
    const jobName = job.name as AttendanceReportJobName;
    switch (jobName) {
      case 'attendance.ENTER':
        this.logger.log(`Attendance Report: Processing ENTER job`);
        return this.handleEnterJob(job);
      case 'attendance.EXIT':
        this.logger.log(`Attendance Report: Processing EXIT job`);
        return this.handleExitJob(job);
      default:
        this.logger.error(`Attendance Report: ${jobName} not found`);
        throw new BadRequestException();
    }
  }

  private async handleEnterJob(job: Job<AttendanceReportJobData>) {
    this.logger.log(`Handling ENTER job for user ${job.data.userId}`);
  }

  private async handleExitJob(job: Job<AttendanceReportJobData>) {
```

```
    this.logger.log(`Handling EXIT job for user ${job.data.userId}`);
    return this.attendanceHandler.handleAttendanceCreated(job.data);
  }
}

import { Injectable, Logger, OnModuleInit } from '@nestjs/common';
import { PrismaService } from 'src/common';

@Injectable()
export class AttendanceSyncService implements OnModuleInit {
  private readonly logger = new Logger(AttendanceSyncService.name);

  constructor(private prismaService: PrismaService) {}

  async onModuleInit() {
    this.logger.log('AttendanceSyncService инициализирован');
    this.logger.log('AttendanceSyncService инициализацияси tugadi');
    // await this.syncAttendanceReportsAll();
  }

  async syncAttendanceReportsAll() {
    this.logger.log('To‘liq sync: barcha Attendance -> AttendanceReport');
    try {
      // Barcha attendance larni olamiz
      const allAttendances = await this.prismaService.attendance.findMany({
        orderBy: [{ userId: 'asc' }, { time: 'asc' }],
      });
      // userId va sana bo‘yicha guruhlash
      const grouped: Record<string, Record<string, { enter: Date[]; exit: Date[] }>> = {};
      for (const att of allAttendances) {
        const userId = att.userId;
        const dateKey = att.time.toISOString().slice(0, 10); // YYYY-MM-DD
        if (!grouped[userId]) grouped[userId] = {};
        if (!grouped[userId][dateKey]) grouped[userId][dateKey] = { enter: [], exit: [] };
```

```
  if (att.type === 'ENTER') grouped[userId][dateKey].enter.push(att.time);
  if (att.type === 'EXIT') grouped[userId][dateKey].exit.push(att.time);
}
let created = 0;
let updated = 0;
let skipped = 0;
for (const userId of Object.keys(grouped)) {
  for (const dateKey of Object.keys(grouped[userId])) {
    const { enter, exit } = grouped[userId][dateKey];
    if (!enter.length) {
      skipped++;
      continue;
    }
    const firstEnter = enter[0];
    const lastExit = exit.length ? exit[exit.length - 1] : firstEnter;
    const minutes = Math.floor((lastExit.getTime() - firstEnter.getTime()) / (1000 * 60));
    // Sana uchun UTC 00:00:00
    const date = new Date(dateKey + 'T00:00:00.000Z');
    // Mavjud reportni topamiz
    const existingReport = await this.prismaService.attendanceReport.findFirst({
      where: { userId, date: { gte: date, lt: new Date(date.getTime() + 24 * 60 * 60 * 1000) } },
    });
    if (existingReport) {
      await this.prismaService.attendanceReport.update({
        where: { id: existingReport.id },
        data: { enter: firstEnter, exit: lastExit, minutes },
      });
      updated++;
    } else {
      await this.prismaService.attendanceReport.create({
        data: { userId, enter: firstEnter, exit: lastExit, date, minutes },
      });
      created++;
    }
```

```
        }
      }
      this.logger.log(`Sync tugadi: created=${created}, updated=${updated}, skipped=${skipped}`);
    } catch (error) {
      this.logger.error(`To‘liq sync xatosi: ${error.message}`, error.stack);
    }
  }
}

import { Injectable, Logger } from '@nestjs/common';
import { Attendance, AttendanceType } from '@prisma/client';
import { PrismaService } from 'src/common';
import * as dayjs from 'dayjs';
import * as utc from 'dayjs/plugin/utc';
import * as timezone from 'dayjs/plugin/timezone';
import * as isBetween from 'dayjs/plugin/isBetween';
import 'dayjs/locale/uz-latn';
import { Dayjs } from 'dayjs';
import { TimezoneHelper, DEFAULT_TIMEZONE } from 'src/helpers/timezone.helper';

dayjs.extend(utc);
dayjs.extend(timezone);
dayjs.extend(isBetween);
dayjs.locale('uz-latn');

dayjs.tz.setDefault(DEFAULT_TIMEZONE);

const extractTimeOnly = (timeValue: any): { hour: number; minute: number } | null => {
  const logger = new Logger('TimeExtractor');

  try {
    if (!timeValue) {
      return null;
    }
```

```
    if (typeof timeValue === 'string') {
      let timeObj = dayjs.utc(timeValue);
      if (timeObj.isValid()) {
        timeObj = timeObj.tz(DEFAULT_TIMEZONE);
        return { hour: timeObj.hour(), minute: timeObj.minute() };
      }
    } else if (timeValue instanceof Date || (typeof timeValue === 'object' && timeValue !== null)) {
      let timeObj = dayjs.utc(timeValue);
      if (timeObj.isValid()) {
        timeObj = timeObj.tz(DEFAULT_TIMEZONE);
        return { hour: timeObj.hour(), minute: timeObj.minute() };
      }
    }

    return null;
  } catch (error) {
    logger.error(`Vaqt ajratishda xatolik: ${error.message}`, error.stack);
    return null;
  }
};

const calculateWorkingMinutes = (
  startTime: number,
  endTime: number,
  breakStartTime: number,
  breakEndTime: number,
): number => {
  const logger = new Logger('WorkingMinutesCalculator');

  try {
    if (startTime >= endTime) {
      return 0;
    }
```

```
    // Tushlik vaqtini hisobga olish
    let breakDuration = 0;
    if (startTime < breakEndTime && endTime > breakStartTime) {
      breakDuration = Math.min(endTime, breakEndTime) - Math.max(startTime, breakStartTime);
      if (breakDuration < 0) breakDuration = 0;
    }

    const workingMinutes = Math.floor((endTime - startTime - breakDuration) / (1000 * 60));
    return workingMinutes;
  } catch (error) {
    logger.error(`Ishlangan vaqtni hisoblashda xatolik: ${error.message}`, error.stack);
    return 0;
  }
};

@Injectable()
export class AttendanceHandler {
  private readonly logger = new Logger(AttendanceHandler.name);

  constructor(private prismaService: PrismaService) {}

  async handleAttendanceCreated(attendance: Attendance): Promise<any> {
    try {
      if (!attendance || !attendance.userId) {
        this.logger.error("Noto'g'ri davomat ma'lumotlari");
        return null;
      }

      this.logger.log(`EXIT event qayta ishlanmoqda: userId=${attendance.userId},
attendanceId=${attendance.id}`);

      // Attendance yozuvining sanasini olish (attendance.time asosida)
      const attendanceDate = TimezoneHelper.toLocalTime(attendance.time);
```

```
    if (!attendanceDate || !attendanceDate.isValid()) {
      this.logger.error(`Noto'g'ri attendance vaqti: ${attendance.time}`);
      return null;
    }

    // DB so'rovlari uchun UTC kun chegaralarini olish
    const { start: startOfDay, end: endOfDay } =
TimezoneHelper.getUTCRangeForLocalDay(attendanceDate);

    // Mahalliy va UTC vaqtlarni logga chiqarish
    this.logger.debug(
      `Hisob-kitob sanasi: ${attendanceDate.format('YYYY-MM-DD')} (${DEFAULT_TIMEZONE}),
UTC: ${dayjs(startOfDay).utc().format('YYYY-MM-DD HH:mm')} -
${dayjs(endOfDay).utc().format('YYYY-MM-DD HH:mm')}`,
    );

    // Kunlik davomat ma'lumotlarini bazadan olish
    const dailyAttendance = await this.getDailyAttendance(attendance.userId, startOfDay, endOfDay);

    this.logger.debug(
      `Kunlik davomat ma'lumotlari: enter=${dailyAttendance.enter ? 'mavjud' : "yo'q"},
exit=${dailyAttendance.exit ? 'mavjud' : "yo'q"}`,
    );

    if (!dailyAttendance.enter || !dailyAttendance.exit) {
      this.logger.warn(
        `Ishlangan vaqtni hisoblash uchun yetarli ma'lumot yo'q: userId=${attendance.userId},
enter=${!!dailyAttendance.enter}, exit=${!!dailyAttendance.exit}`,
      );
      return null;
    }

    // Foydalanuvchi ish jadvalini olish
    const userSchedule = await this.getUserWorkingSchedule(attendance.userId, attendanceDate);
```

```
      // Tushlik vaqtini olish
      const breakTime = await this.getBreakTime(attendance.userId, attendanceDate);

      // Ishlangan vaqtni hisoblash
      const workingTimeResult = this.calculateWorkingTime(
        dailyAttendance.enter,
        dailyAttendance.exit,
        userSchedule,
        breakTime,
        attendanceDate,
      );

      this.logger.debug('WorkingTimeResult: ', workingTimeResult);

      // Hisobotni yaratish yoki yangilash
      return this.createOrUpdateAttendanceReport(attendance.userId, dailyAttendance,
workingTimeResult, attendanceDate);
    } catch (error) {
      this.logger.error(`EXIT hodisasini qayta ishlashda xatolik: ${error.message}`, error.stack);
      return null;
    }
  }

  private async getDailyAttendance(userId: string, startOfDay: Date, endOfDay: Date) {
    try {
      // Birinchi kirish vaqti
      const firstEnterAttendance = await this.prismaService.attendance.findFirst({
        where: {
          userId,
          type: AttendanceType.ENTER,
          time: {
            gte: startOfDay,
            lt: endOfDay,
          },
```

```
  },
  orderBy: { time: 'asc' },
});

// Oxirgi chiqish vaqti
const lastExitAttendance = await this.prismaService.attendance.findFirst({
  where: {
    userId,
    type: AttendanceType.EXIT,
    time: {
      gte: startOfDay,
      lt: endOfDay,
    },
  },
  orderBy: { time: 'desc' },
});

// Kunlik barcha kirish-chiqishlar
const allAttendances = await this.prismaService.attendance.findMany({
  where: {
    userId,
    time: {
      gte: startOfDay,
      lt: endOfDay,
    },
  },
  orderBy: { time: 'asc' },
});

return {
  enter: firstEnterAttendance?.time || null,
  exit: lastExitAttendance?.time || null,
  allAttendances, // Keyingi versiyalarda butun kun bo'yicha analiz qilish uchun
};
```

```
  } catch (error) {
    this.logger.error(`Kunlik davomat ma'lumotlarini olishda xatolik: ${error.message}`, error.stack);
    return { enter: null, exit: null, allAttendances: [] };
  }
}

private async getUserWorkingSchedule(userId: string, today: Dayjs) {
  try {
    const userWorkingSchedule = await this.prismaService.userWorkingSchedule.findUnique({
      where: { userId },
      select: {
        days: {
          select: {
            day: true,
            startTime: true,
            endTime: true,
          },
        },
      },
    });

    if (!userWorkingSchedule) {
      return null;
    }

    const todayNumber = today.day();
    const day = userWorkingSchedule.days.find((d) => d.day === todayNumber);

    if (!day) {
      return null;
    }

    // Jadval vaqtlarini ajratib olish
    const startTimeObj = extractTimeOnly(day.startTime);
```

```
      const endTimeObj = extractTimeOnly(day.endTime);

      if (!startTimeObj || !endTimeObj) {
        return null;
      }

      // Vaqtlarni UTC joriy kunga o'rnatish
      const scheduledStart =
today.hour(startTimeObj.hour).minute(startTimeObj.minute).second(0).millisecond(0);

      const scheduledEnd =
today.hour(endTimeObj.hour).minute(endTimeObj.minute).second(0).millisecond(0);
      return {
        start: scheduledStart,
        end: scheduledEnd,
      };
    } catch (error) {
      this.logger.error(`Foydalanuvchi jadvalini olishda xatolik: ${error.message}`, error.stack);
      return null;
    }
  }

  private async getBreakTime(userId: string, today: Dayjs) {
    try {
      const breakStart: Dayjs = today.hour(13).minute(0).second(0).millisecond(0);
      const breakEnd: Dayjs = today.hour(14).minute(0).second(0).millisecond(0);

      return {
        start: breakStart,
        end: breakEnd,
      };
    } catch (error) {
      this.logger.error(`Tushlik vaqtini olishda xatolik: ${error.message}`, error.stack);
```

```
    // Xatolik yuz berganda standart tushlik vaqti
    return {
      start: today.hour(13).minute(0).second(0).millisecond(0),
      end: today.hour(14).minute(0).second(0).millisecond(0),
    };
  }
}

private calculateWorkingTime(
  enterTime: Date,
  exitTime: Date,
  userSchedule: { start: Dayjs; end: Dayjs } | null,
  breakTime: { start: Dayjs; end: Dayjs },
  today: Dayjs,
) {
  try {
    const enterTimeMs = enterTime.getTime();
    const exitTimeMs = exitTime.getTime();
    const breakStartMs = breakTime.start.valueOf();
    const breakEndMs = breakTime.end.valueOf();

    let actualWorkingMinutes = 0;
    let scheduledWorkingMinutes = 0;
    let underTimeMinutes = 0;
    let isLate = false;
    let isEarlyExit = false;
    let lateEnter = null;
    let earlyExit = null;
    let scheduledEnter = null;
    let scheduledExit = null;

    // Ish jadvali mavjud bo'lsa
    if (userSchedule) {
      scheduledEnter = userSchedule.start.toDate();
```

```
scheduledExit = userSchedule.end.toDate();
const scheduledStartMs = userSchedule.start.valueOf();
const scheduledEndMs = userSchedule.end.valueOf();

// Tushlik vaqti ish jadvali bilan kesishadimi
const breakOverlapsWithSchedule = breakStartMs < scheduledEndMs && breakEndMs >
scheduledStartMs;

// Rejalashtirilgan ish vaqti (tushlik vaqtini ayirib)
scheduledWorkingMinutes = Math.floor(
  (scheduledEndMs - scheduledStartMs - (breakOverlapsWithSchedule ? breakEndMs -
breakStartMs : 0)) /
    (1000 * 60),
);

// Haqiqiy ish vaqti chegaralari
const actualStartMs = Math.max(enterTimeMs, scheduledStartMs);
const actualEndMs = Math.min(exitTimeMs, scheduledEndMs);

// Haqiqiy ishlangan vaqt
if (actualStartMs < actualEndMs) {
  actualWorkingMinutes = calculateWorkingMinutes(actualStartMs, actualEndMs, breakStartMs,
breakEndMs);
}

// Kechikish va erta ketishni hisoblash
if (enterTimeMs > scheduledStartMs) {
  isLate = true;
  lateEnter = Math.floor((enterTimeMs - scheduledStartMs) / (1000 * 60));
}

if (exitTimeMs < scheduledEndMs) {
  isEarlyExit = true;
  earlyExit = Math.floor((scheduledEndMs - exitTimeMs) / (1000 * 60));
}
```

```
      // Kam ishlangan vaqt
      if (scheduledWorkingMinutes > 0 && actualWorkingMinutes < scheduledWorkingMinutes) {
        underTimeMinutes = scheduledWorkingMinutes - actualWorkingMinutes;
      }
    } else {
      // Ish jadvali mavjud bo'lmaganda
      actualWorkingMinutes = calculateWorkingMinutes(enterTimeMs, exitTimeMs, breakStartMs,
breakEndMs);
    }

    return {
      actualWorkingMinutes,
      scheduledWorkingMinutes: scheduledWorkingMinutes || null,
      underTimeMinutes: underTimeMinutes || null,
      isLate,
      isEarlyExit,
      lateEnter,
      earlyExit,
      scheduledEnter,
      scheduledExit,
    };
  } catch (error) {
    this.logger.error(`Ishlangan vaqtni hisoblashda xatolik: ${error.message}`, error.stack);
    return {
      actualWorkingMinutes: 0,
      scheduledWorkingMinutes: null,
      underTimeMinutes: null,
      isLate: false,
      isEarlyExit: false,
      lateEnter: null,
      earlyExit: null,
      scheduledEnter: null,
      scheduledExit: null,
```

```
    };
  }
}

private async createOrUpdateAttendanceReport(
  userId: string,
  dailyAttendance: { enter: Date; exit: Date },
  workingTimeResult: any,
  today: Dayjs,
) {
  try {
    const { start: startOfDay, end: endOfDay } = TimezoneHelper.getUTCRangeForLocalDay(today);
    const dayNumber = today.day() === 0 ? 7 : today.day();

    const existingReport = await this.prismaService.attendanceReport.findFirst({
      where: {
        userId,
        date: {
          gte: startOfDay,
          lt: endOfDay,
        },
      },
    });

    const reportData = {
      userId,
      enter: dailyAttendance.enter,
      exit: dailyAttendance.exit,
      date: today.toISOString(),
      dayNumber,
      minutes: workingTimeResult.actualWorkingMinutes,
      scheduledMinutes: workingTimeResult.scheduledWorkingMinutes,
      underTimeMinutes: workingTimeResult.underTimeMinutes,
      isLate: workingTimeResult.isLate,
```

```
        isEarlyExit: workingTimeResult.isEarlyExit,
        lateEnter: workingTimeResult.lateEnter,
        earlyExit: workingTimeResult.earlyExit,
        scheduledEnter: workingTimeResult.scheduledEnter,
        scheduledExit: workingTimeResult.scheduledExit,
      };

      if (!existingReport) {
        const newReport = await this.prismaService.attendanceReport.create({
          data: reportData,
        });
        this.logger.log(`Hisobot yaratildi: userId=${userId}, reportId=${newReport.id}`);
        return newReport;
      } else {
        const updatedReport = await this.prismaService.attendanceReport.update({
          where: { id: existingReport.id },
          data: reportData,
        });
        this.logger.log(`Hisobot yangilandi: userId=${userId}, reportId=${updatedReport.id}`);
        return updatedReport;
      }
    } catch (error) {
      this.logger.error(`Hisobotni yaratish yoki yangilashda xatolik: ${error.message}`, error.stack);
      return null;
    }
  }
}

import { Module } from '@nestjs/common';
import { AttendanceService } from './attendance.service';
import { AttendanceController } from './attendance.controller';
import { BullModule } from '@nestjs/bullmq';
import { ATTENDANCE, ATTENDANCE_REPORT } from './interface/attendance';
import { AttendanceProcessor } from './attendance.processor';
```

```
import { BullBoardModule } from '@bull-board/nestjs';
import { BullMQAdapter } from '@bull-board/api/bullMQAdapter';
import { AttendanceHandler } from './attendance.handler';
import { AttendanceExcelService } from './attendace-excel.service';
import { AttendanceBasicService } from './services/attendance-basic.service';
import { AttendanceHelperService } from './services/attendance-helper.service';
import { AttendanceTableService } from './services/attendance-table.service';
import { AttendanceReportService } from './services/attendance-report.service';
import { AttendanceEmployeeReportService } from './services/attendance-employee-report.service';
import { AttendanceReportProcessor } from './attendance-report.processor';

@Module({
  imports: [
    BullModule.registerQueue({
      name: ATTENDANCE,
      defaultJobOptions: {
        removeOnComplete: 100,
        removeOnFail: 50,
        attempts: 3,
        backoff: {
          type: 'exponential',
          delay: 2000,
        },
      },
    }),
    BullModule.registerQueue({
      name: ATTENDANCE_REPORT,
      defaultJobOptions: {
        removeOnComplete: 100,
        removeOnFail: 50,
        attempts: 3,
        backoff: {
          type: 'exponential',
          delay: 2000,
```

```
        },
      },
    }),
    BullBoardModule.forFeature({
      name: ATTENDANCE,
      adapter: BullMQAdapter,
    }),
    BullBoardModule.forFeature({
      name: ATTENDANCE_REPORT,
      adapter: BullMQAdapter,
    }),
  ],
  controllers: [AttendanceController],
  providers: [
    AttendanceService,
    AttendanceBasicService,
    AttendanceHelperService,
    AttendanceTableService,
    AttendanceReportService,
    AttendanceEmployeeReportService,
    AttendanceProcessor,
    AttendanceHandler,
    AttendanceExcelService,
    AttendanceReportProcessor,
  ],
})
export class AttendanceModule {}


import { InjectQueue, Processor, WorkerHost } from '@nestjs/bullmq';
import { ATTENDANCE, ATTENDANCE_REPORT, AttendanceData } from './interface/attendance';
import { Job, Queue } from 'bullmq';
import { BadRequestException, Inject, Injectable, Logger, NotFoundException } from
'@nestjs/common';
import { PrismaService } from 'src/common';
```

```
import { EventEmitter2 } from '@nestjs/event-emitter';
import { AttendanceType } from '@prisma/client';
import * as dayjs from 'dayjs';
import { Dayjs } from 'dayjs';
import * as utc from 'dayjs/plugin/utc';
import * as timezone from 'dayjs/plugin/timezone';
import * as duration from 'dayjs/plugin/duration';
import 'dayjs/locale/uz-latn';
import { TimezoneHelper } from 'src/helpers/timezone.helper';

// Dayjs pluginlarini o'rnatish
dayjs.extend(utc);
dayjs.extend(timezone);
dayjs.extend(duration);
dayjs.locale('uz-latn');

// Standart vaqt zonasini belgilash
const DEFAULT_TIMEZONE = 'Asia/Tashkent';
dayjs.tz.setDefault(DEFAULT_TIMEZONE);

const parseDateTime = (dateTimeValue: string | Date): Dayjs | null => {
  return TimezoneHelper.parseDateTime(dateTimeValue);
};

@Injectable()
@Processor(ATTENDANCE, {
  concurrency: 4,
  limiter: {
    max: 50,
    duration: 1000,
  },
})
export class AttendanceProcessor extends WorkerHost {
  private readonly logger = new Logger(AttendanceProcessor.name);
```

```
  constructor(
    private prismaService: PrismaService,
    private readonly eventEmitter: EventEmitter2,
    @InjectQueue(ATTENDANCE_REPORT) private attendanceReportQueue: Queue,
  ) {
    super();
  }

  async process(job: Job<AttendanceData>, token?: string) {
    try {
      this.logger.log(`Davomat ma'lumotini qayta ishlash boshlandi: jobId=${job.id}`);

      if (!this.validateJobData(job.data)) {
        throw new BadRequestException("Noto'g'ri job ma'lumotlari");
      }

      const { user, device } = await this.validateUserAndDevice(job.data);

      const eventDateUTC = parseDateTime(job.data.dateTime);
      if (!eventDateUTC) {
        throw new BadRequestException("Noto'g'ri sana va vaqt formati");
      }

      this.logger.debug(
        `Davomat eventi: userId=${user.id}, deviceMAC=${device.MAC},
vaqt=${eventDateUTC.toISOString()} (UTC)`,
      );

      const dailyAttendanceState = await this.analyzeDailyAttendanceState(user.id, eventDateUTC);

      const result = await this.processAttendanceRecord(
        user.id,
        device.organizationId,
```

```
      eventDateUTC,
      dailyAttendanceState,
    );

    return result;
  } catch (error) {
    this.logger.error(`Davomatni qayta ishlashda xatolik: ${error.message}`, error.stack);
    throw error;
  }
}

private validateJobData(data: AttendanceData): boolean {
  if (!data) {
    this.logger.error("Job ma'lumotlari bo'sh");
    return false;
  }

  if (!data.AccessControllerEvent?.employeeNoString) {
    this.logger.error('Xodim ID si mavjud emas');
    return false;
  }

  if (!data.macAddress) {
    this.logger.error('Qurilma MAC adresi mavjud emas');
    return false;
  }

  if (!data.dateTime) {
    this.logger.error('Sana va vaqt mavjud emas');
    return false;
  }

  return true;
}
```

```
private async validateUserAndDevice(data: AttendanceData) {
  try {
    const user = await this.prismaService.user.findUnique({
      where: { id: data.AccessControllerEvent.employeeNoString },
      select: {
        id: true,
        fullName: true,
      },
    });

    if (!user) {
      this.logger.error(`Foydalanuvchi topilmadi: id=${data.AccessControllerEvent.employeeNoString}`);
      throw new NotFoundException('Foydalanuvchi topilmadi');
    }

    const device = await this.prismaService.faceIDDevice.findUnique({
      where: { MAC: data.macAddress },
      select: {
        id: true,
        MAC: true,
        organizationId: true,
      },
    });

    if (!device) {
      this.logger.error(`Face ID qurilma topilmadi: MAC=${data.macAddress}`);
      throw new NotFoundException('Face ID qurilma topilmadi');
    }

    return { user, device };
  } catch (error) {
    this.logger.error(`Foydalanuvchi yoki qurilmani tekshirishda xatolik: ${error.message}`);
    throw error;
```

```
    }
  }

  private async analyzeDailyAttendanceState(userId: string, eventDateUTC: Dayjs) {
    try {
      const { start: dayRangeStart, end: dayRangeEnd } =
TimezoneHelper.getUTCRangeForLocalDay(eventDateUTC);

      const dailyAttendances = await this.prismaService.attendance.findMany({
        where: {
          userId,
          time: {
            gte: dayRangeStart,
            lt: dayRangeEnd,
          },
        },
        orderBy: { time: 'asc' },
        select: {
          id: true,
          type: true,
          time: true,
        },
      });

      const enterRecords = dailyAttendances.filter((a) => a.type === AttendanceType.ENTER);
      const exitRecords = dailyAttendances.filter((a) => a.type === AttendanceType.EXIT);

      const lastEnterRecord = enterRecords.length > 0 ? enterRecords[enterRecords.length - 1] : null;
      const lastExitRecord = exitRecords.length > 0 ? exitRecords[exitRecords.length - 1] : null;

      return {
        dailyAttendances,
        enterRecords,
        exitRecords,
```

```
      lastEnterRecord,
      lastExitRecord,
      dayRange: { start: dayRangeStart, end: dayRangeEnd },
    };
  } catch (error) {
    this.logger.error(`Kunlik davomat holatini tahlil qilishda xatolik: ${error.message}`);
    throw error;
  }
}

private async processAttendanceRecord(userId: string, organizationId: string, eventDateUTC: Dayjs,
dailyState: any) {
  try {
    const eventTimeISO = eventDateUTC.toISOString();

    if (dailyState.dailyAttendances.length === 0) {
      const enterRecords = await this.createEnterRecord(userId, organizationId, eventTimeISO);
      const exitRecords = await this.createExitRecord(userId, organizationId, eventTimeISO);
      return [...enterRecords, ...exitRecords];
    }

    const firstAttendance = dailyState.dailyAttendances[0];
    if (firstAttendance.type !== AttendanceType.ENTER) {
      const updatedAttendance = await this.prismaService.attendance.update({
        where: { id: firstAttendance.id },
        data: { type: AttendanceType.ENTER },
      });

      await this.emitEventSafely('attendance.ENTER', updatedAttendance);
      return await this.createExitRecord(userId, organizationId, eventTimeISO);
    }

    return await this.createExitRecord(userId, organizationId, eventTimeISO);
  } catch (error) {
```

```
    this.logger.error(`Davomat yozuvini qayta ishlashda xatolik: ${error.message}`);
    throw error;
  }
}

private async createEnterRecord(userId: string, organizationId: string, eventTimeISO: string) {
  const user = await this.prismaService.user.findUnique({
    where: { id: userId },
    select: {
      fullName: true,
      Organization: {
        select: {
          id: true,
          name: true,
        },
      },
    },
  });

  try {
    const attendanceRecords = await this.prismaService.$transaction(async (tx) => {
      return await Promise.all(
        user.Organization.map(async (org) => {
          return tx.attendance.create({
            data: {
              userId,
              organizationId: org.id,
              time: eventTimeISO,
              type: AttendanceType.ENTER,
            },
          });
        }),
      );
    });
```

```
    for (const attendance of attendanceRecords) {
      await this.emitEventSafely('attendance.ENTER', attendance);
    }

    return attendanceRecords;
  } catch (error) {
    this.logger.error(`ENTER yozuvini yaratishda xatolik: ${error.message}`);
    throw error;
  }
}

private async createExitRecord(userId: string, organizationId: string, eventTimeISO: string) {
  const user = await this.prismaService.user.findUnique({
    where: { id: userId },
    select: {
      fullName: true,
      Organization: {
        select: {
          id: true,
          name: true,
        },
      },
    },
  });

  try {
    const attendanceRecords = await this.prismaService.$transaction(async (tx) => {
      return Promise.all(
        user.Organization.map(async (org) => {
          const attendance = await tx.attendance.create({
            data: {
              userId,
              organizationId: org.id,
```

```
            time: eventTimeISO,
            type: AttendanceType.EXIT,
          },
        });
        return attendance;
      }),
    );
  });

  for (const attendance of attendanceRecords) {
    await this.emitEventSafely('attendance.EXIT', attendance);
  }

  this.logger.log(`EXIT yozuvlari muvaffaqiyatli yaratildi: userId=${userId},
soni=${attendanceRecords.length}`);
  return attendanceRecords;
 } catch (error) {
  this.logger.error(`EXIT yozuvini yaratishda xatolik: ${error.message}`);
  throw error;
 }
}

private async emitEventSafely(event: string, data: any) {
 try {
  await new Promise((resolve) => setTimeout(resolve, 10));
  await this.attendanceReportQueue.add(event, data);
 } catch (error) {
  this.logger.error(`Event emit xatoligi: ${event}`, error.message);
 }
}
}

import { Injectable, NotFoundException } from '@nestjs/common';
import { CreateAttendanceDto, FindAttendanceDto } from './dto/create-attendance.dto';
```

```
import { AttendanceReportQueryDto } from './dto/attendance-report.dto';
import {
  AttendanceBasicService,
  AttendanceTableService,
  AttendanceReportService,
  AttendanceEmployeeReportService,
} from './services';
import { PrismaService } from 'src/common';

@Injectable()
export class AttendanceService {
  constructor(
    private basicService: AttendanceBasicService,
    private tableService: AttendanceTableService,
    private reportService: AttendanceReportService,
    private employeeReportService: AttendanceEmployeeReportService,
    private prismaService: PrismaService,
  ) {}

  // Базовые операции
  async createAttendance(data: Omit<CreateAttendanceDto, 'time' | 'description'>) {
    return this.basicService.createAttendance(data);
  }

  async getUsersByPosition(positionId: string) {
    const position = await this.prismaService.organizationTypePosition.findUnique({
      where: { id: positionId },
    });

    if (!position) {
      throw new NotFoundException(`Position with id ${positionId} not found`);
    }

    const users = await this.prismaService.user.findMany({
```

```
    where: {
      positionId: positionId,
      status: 'ACTIVE',
    },
    select: {
      id: true,
      fullName: true,
      positionId: true,
    },
  });

  return users;
}

async getUserAttendance(userId: string, page: string, limit: string) {
  return this.basicService.getUserAttendance(userId, page, limit);
}

async getAttendance(id: string, query: FindAttendanceDto) {
  return this.basicService.getAttendance(id, query);
}

// Табличные отчеты
async getAttendanceTable(id: string, query: FindAttendanceDto) {
  return this.tableService.getAttendanceTable(id, query);
}

// Отчеты по организациям
async getAttendanceReport(orgIds: string[], startDate?: string, endDate?: string) {
  return this.reportService.getAttendanceReport(orgIds, startDate, endDate);
}

async getAttendanceReportByOrganizationTypePosition(
  orgId: string,
```

```
    positionId: string,
    startDate?: string,
    endDate?: string,
    page: number = 1,
    limit: number = 20,
  ) {
    return this.reportService.getAttendanceReportByOrganizationTypePosition(
      orgId,
      positionId,
      startDate,
      endDate,
      page,
      limit,
    );
  }

  async getOptimizedAttendanceReport(query: AttendanceReportQueryDto) {
    return this.reportService.getOptimizedAttendanceReport(query);
  }

  // Детальные отчеты по сотрудникам
  async getEmployeeDetailedAttendanceReport(query: AttendanceReportQueryDto) {
    return this.employeeReportService.getEmployeeDetailedAttendanceReport(query);
  }
}
```